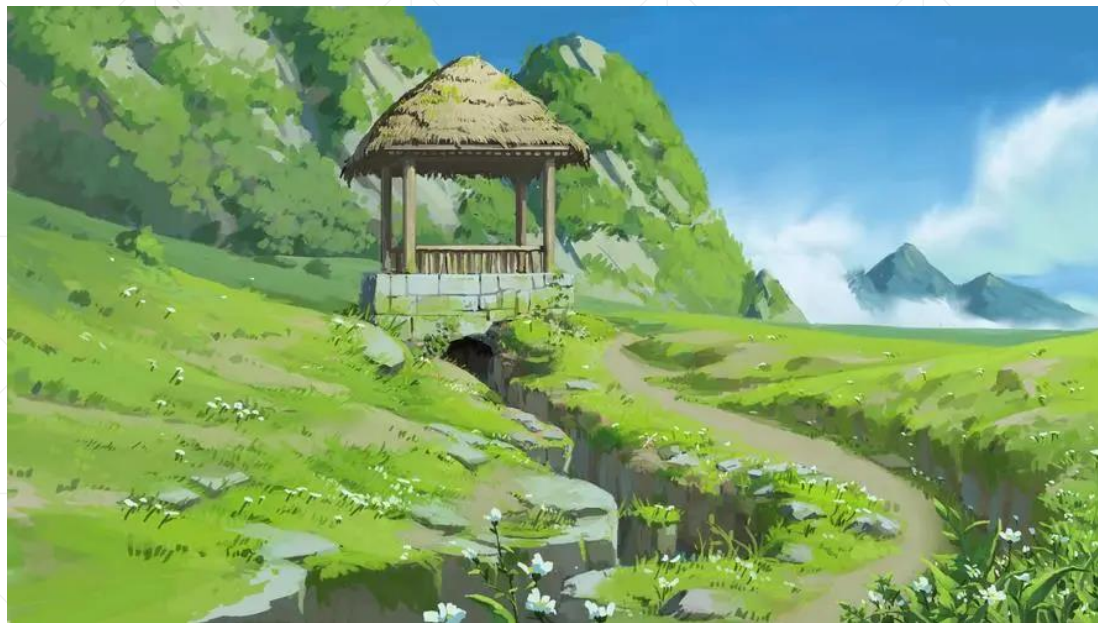




Scene与Node

北京理工大学计算机学院
金旭亮

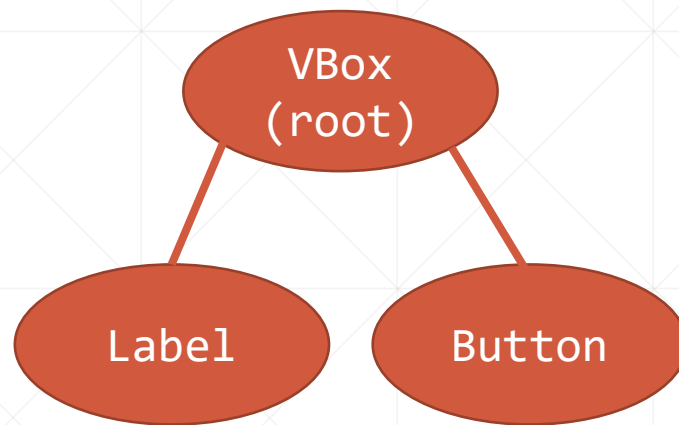
控件 (Control) 与场景 (Scene)



示例: SceneGraph.java

本讲通过一个非常简单的示例，介绍JavaFX中两个核心组件——Scene和Control。

```
public void start(Stage primaryStage) {  
    //标签用于显示信息  
    Label message = new Label( text: "Hello, JavaFX!");  
    message.setFont(new Font( size: 40));  
  
    //按钮支持事件响应  
    Button red = new Button( text: "字体变红");  
    //使用Lambda表达式定义按钮单击事件响应代码  
    red.setOnAction(event -> message.setTextFill(Color.RED));  
  
    //VBox是一个布局 (Layout) 控件，它按照从上到下的顺序排列多个控件  
    VBox root = new VBox();  
    root.setPadding(new Insets( topRightBottomLeft: 10));  
    root.setAlignment(Pos.CENTER);  
    root.setSpacing(10);  
    //将按钮和标签加入到VBox中，先加入的排在上面  
    root.getChildren().addAll(red, message);  
  
    //构建场景并显示  
    Scene scene = new Scene(root);  
    primaryStage.setScene(scene);  
    primaryStage.setTitle("SceneGraph");  
    primaryStage.show();  
}
```

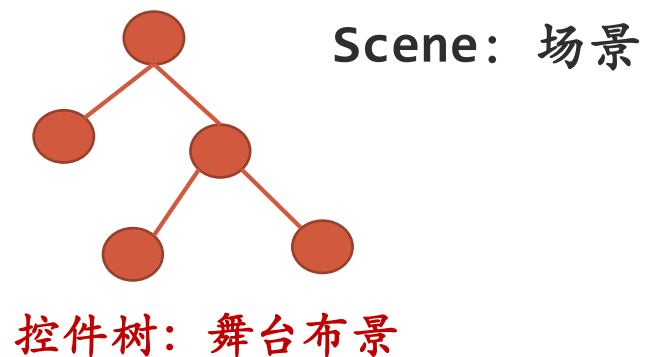


示例构建了一棵控件树，每个节点都是一个node对象。

这棵控件树，关联上Scene之后，称为“场景图 (Scene Graph)”。

示例要点

Stage: 舞台



⚡ 控件树中的节点支持事件响应。

在JavaFX中，Scene中的内容会以由节点(Node)构成的分层**场景图(Scene Graph)**来展现。

Scene Graph其实就是一棵多叉树，各种控件都是树中的节点。

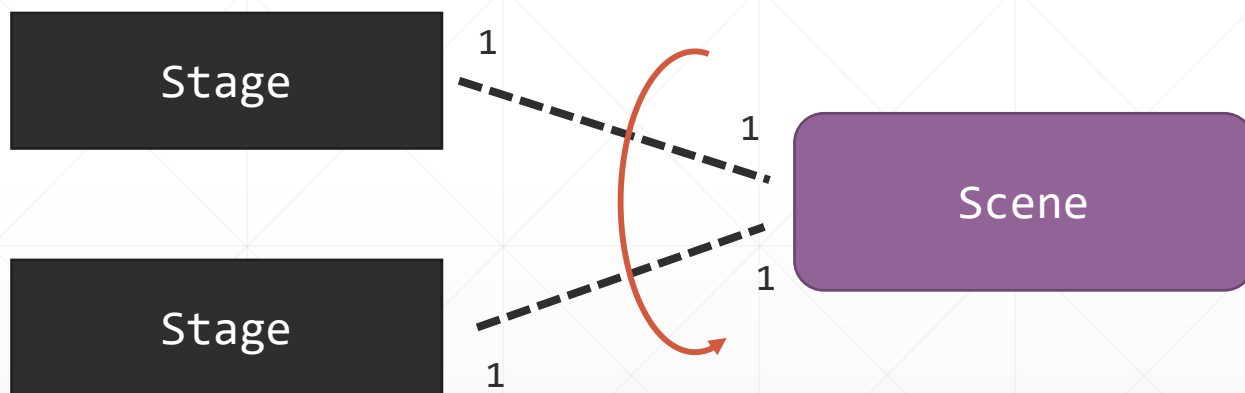
最底层节点通常都是诸如按钮之类的控件，也可以是Circle之类的图形。

拥有子树的中间节点对象称为“容器”，它们很多都是“**布局(Layout)**”控件。

Stage与Scene



一个Stage一次只能关联一个Scene，一个Scene一次只能“属于”同一个Stage。



如果一个Scene需要关联另一个Stage，必须先移除（detach）它与当前Stage的关联，再关联到另一个Stage。

关于Node



场景图（Scene Graph）中的基本构成单元是Node对象。



Node具有以下公共的特性：

1

能响应用户键盘和鼠标操作

2

JavaFX提供了一些图形特效（effects）和变换（Transform），可用应用于Node对象

3

Node拥有自己的状态，其状态改变时，往往会导致界面的重绘。

Node类型



Node是JavaFX的顶层基类，是一个抽象类，所有控件都间接地派生自它，比如，按钮控件（Button），就是Node类的子类型。

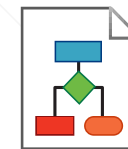


一些Node对象，可以包容另外一些Node对象，这些Node对象，其实就是“容器控件”（其中很多都是“布局控件”），JavaFX中最常见的容器控件，就是各种各样的面板控件，比如“GridPane（网格面板）”。



所有拥有儿子的节点，都直接或间接地派生自Parent类。

Node的访问方式与场景图的修改



Node实例只能出现在“场景图（Scene Graph）”中的一个地方，也就是说，同一个Node对象，不能加入两次到场景图中。



构成场景图中的Node，可以给它分配一个唯一的id值，之后在控制器中就可以通过这个id值访问到这个Node对象，这就是JavaFX应用普遍采用的MVC编程模式，将访问控件对象的代码集中到控制器中。



仅仅只有JavaFX Application Thread可以修改场景图，不允许跨线程直接访问已经加入到场景图中的节点对象。

给Node添加子节点



✓ 对于那些拥有子节点的Node类型（比如布局控件），调用 `getChildren()` 获取子节点集合，然后再调用 `add()` 或 `addAll()` 方法添加子节点，这是一种通用的编程模式。

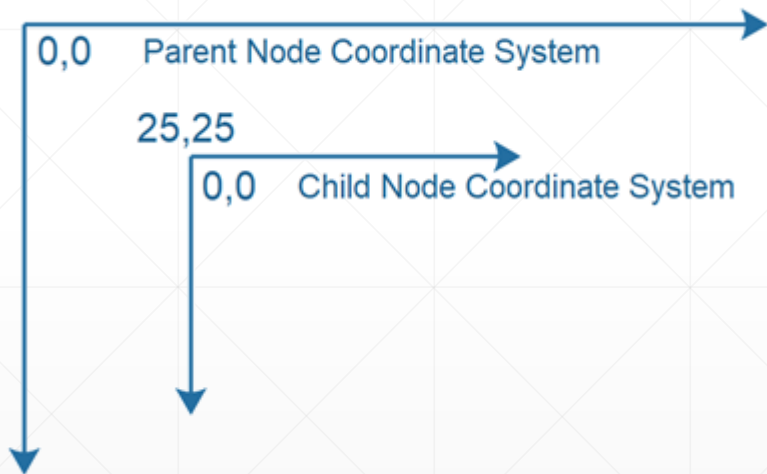
✓ 子节点添加完成之后，界面会自动地刷新，这个是由JavaFX运行时自动地完成的。

JavaFX会跟踪当前Node对象的状态，当发现其状态有变化（其实就是Node的相关属性值有变化）时，会自动地“重绘”有变化的部分。这部分工作对于程序员来说是“透明”的。

Node的定位与布局



每个Node拥有自己的坐标系，基于它来绘制自己，但其范围受限于它的Parent Node。



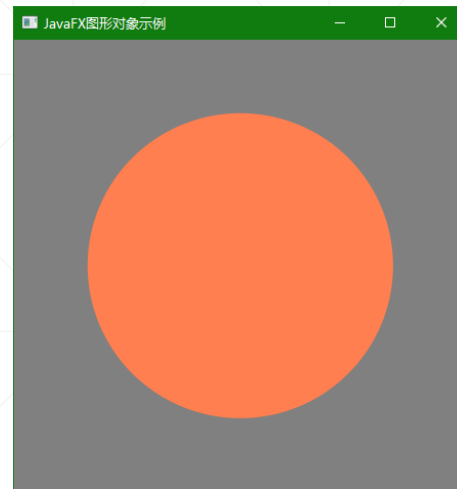
可以使用`setLayoutX()`和`setLayoutY()`来定位子控件（以父控件的坐标系为基准），但这个调用是否生效，取决于父节点的具体类型是什么。



Node定义了`preferredWidth`和`preferredHeight`两个属性，用于声明它期望拥有的尺寸，父节点可能会调整其子节点的宽度和高度值，从而导致代码中对特定Node对象这两个字段值的设置失效。

Shape也是一种Node的子类型

```
public void start(Stage primaryStage) {  
    //Group是一个控件容器  
    Group root = new Group();  
    //将容器对象加入到场景中  
    Scene scene = new Scene(root, width: 400, height: 400, Color.GRAY);  
    //构建一个圆对象  
    Circle cir = new Circle(centerX: 200, centerY: 200, radius: 50);  
    cir.setFill(Color.CORAL);  
    //让圆对象响应鼠标点击事件，每点击一次，半径加5  
    cir.setOnMouseClicked(e->cir.setRadius(cir.getRadius()+5));  
    //将圆对象加入到容器对象中  
    root.getChildren().add(cir);  
    //设置“舞台”，并显示在屏幕上  
    primaryStage.setTitle("JavaFX图形对象示例");  
    primaryStage.setScene(scene);  
    primaryStage.show();  
}
```



示例：JavaFXShape.java

JavaFX支持将一些基础图形（Shape类的子类）加入到控件树中，可以使用这些图形构建数据可视化应用（比如各种图表）

Node对象另外一些通用特性



Node支持添加边框，并且可以设定其颜色、宽度和线型。



JavaFX中的Node对象，可以添加各种特效，比如加上阴影和模糊效果，通过调用Opacity值，Node对象支持实现“半透明”效果。

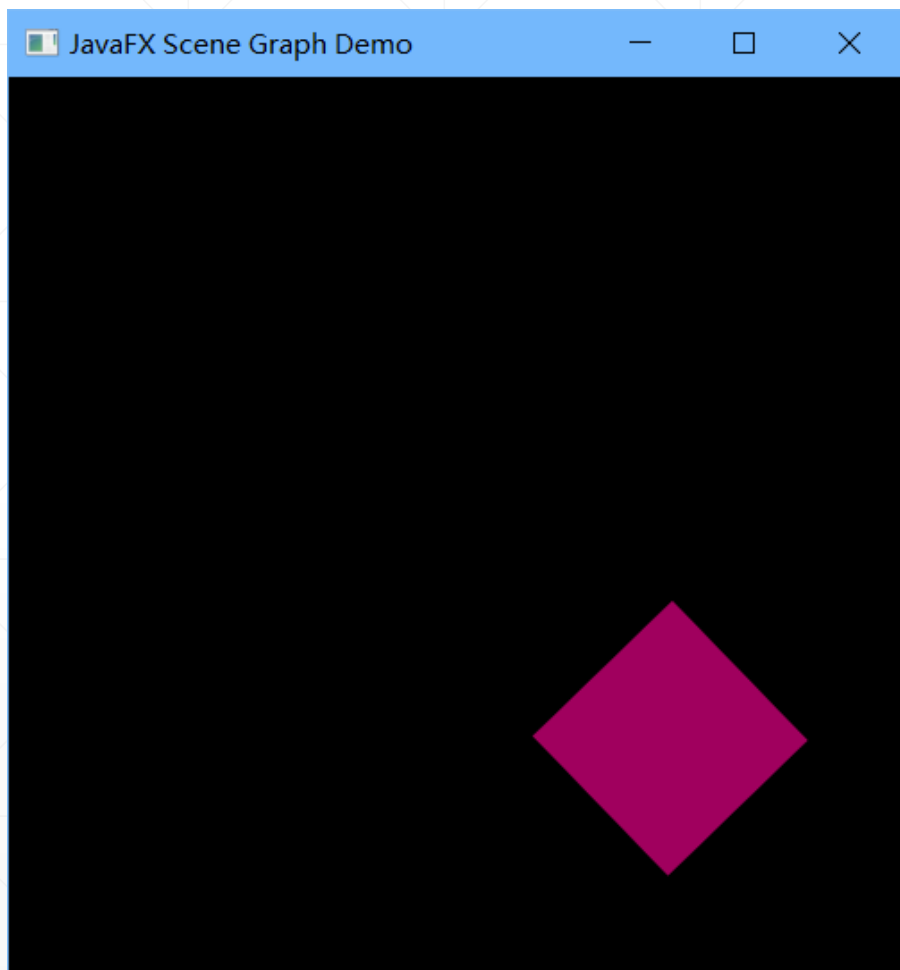


Node对象可以通过各种变换操作，在底层利用GPU的图形功能，实现平滑的动画。



Node对象可以响应鼠标和键盘操作，并且可以保存状态。

多媒体与动画



JavaFX内置各种动画效果（通过一组Transition类型实现），可以很方便地应用于JavaFX的图形与控件上。

示例：Animation.java

给Node附加用户数据



Node定义了一个setUserData()方法，可以使用它为节点附加一些额外的数据，比如，与此节点相关联的JavaBean。

```
node.setUserData(new MyObject("Hey - some data"));
```

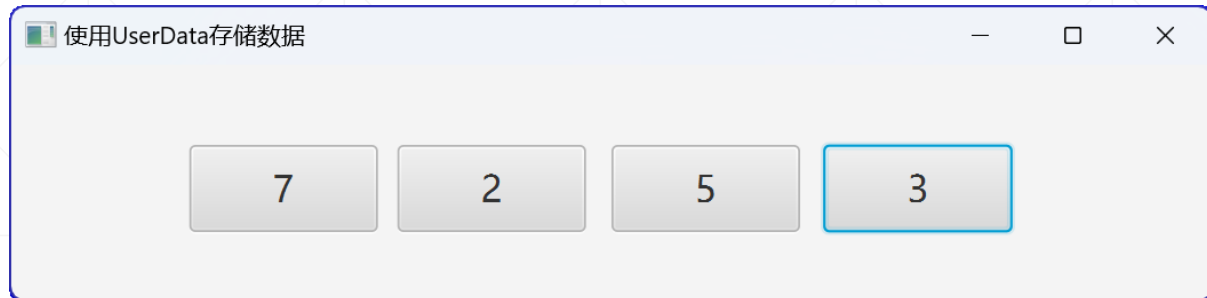


这么做了之后，在需要时，可以调用getUserData()方法取出这些数据。



由于所有控件都是Node的子类，因此，userData总是可用的，记住这个特性，在实际开发中在合适的地方用上它，有奇效。

UserData示例



四个按钮，每个按钮都携带一个“计数器”，其值保存到按钮对象的userData中。

UserData示例分析



@Override

```
public void initialize(URL url, ResourceBundle resourceBundle) {
```

```
    btn1.setUserData(0);  
    btn2.setUserData(0);  
    btn3.setUserData(0);  
    btn4.setUserData(0);
```

设置初始值

```
    btn1.setOnAction(this::clickButton);  
    btn2.setOnAction(this::clickButton);  
    btn3.setOnAction(this::clickButton);  
    btn4.setOnAction(this::clickButton);
```

```
}
```

取出保存的值，修改之后，再保存回去.....

4 usages

```
private void clickButton(ActionEvent e) {  
    var button = (Button) e.getSource();  
    Integer currentCount = (Integer) button.getUserData();  
    currentCount += 1;  
    button.setUserData(currentCount);  
    button.setText(currentCount.toString());
```

```
}
```


Node数据绑定



JavaFX中许多Node类定义的属性，是一种特殊类型的属性，称为“**JavaFX Bean Property**”，这种属性在值改变时会触发“值改变”通知，从而可以让界面自动更新。

基于JavaFX Bean Property，可以在两个Node对象的两个属性之间建立一种“绑定”关系，一个Node对象特定属性值的变化，会触发另一个Node对象相关属性值的变化。这种编程技巧，在实际开发中用得非常普遍。

Databind示例分析

```
@Override
public void start(Stage primaryStage) {
    //创建两个标签对象
    Label info = new Label( text: "Label字体大小绑定到Slider的值");
    info.setFont(new Font( size: 20));
    Label message = new Label( text: "Hello, JavaFX!");
    message.setFont(new Font( size: 50));
    //设置Slider控件的相关属性
    Slider slider = new Slider();
    slider.setMax(50);
    slider.setMin(10);
    slider.setValue(50);

    //在Slider对象和Label对象之间构建数据绑定关系
    slider.valueProperty().addListener(property
        -> message.setFont(new Font(slider.getValue())));
}
```

通过附加监听器，实现了两个控件间对应属性值的状态同步



```
//构建场景图
VBox root = new VBox();
root.setSpacing(10);
root.setPadding(new Insets( topRightBottomLeft: 20));
root.setAlignment(Pos.CENTER);
root.getChildren().addAll(info, slider, message);
```

```
//显示Stage
Scene scene = new Scene(root, width: 400, height: 200);
primaryStage.setTitle("Node对象之间的数据绑定");
primaryStage.setScene(scene);
primaryStage.show();
```

```
}
```